8glbibh0p

January 15, 2025

[7]:
```
!pip install -q mediapipe
```

```
                         35.6/35.6 MB
28.3 MB/s eta 0:00:00
```

[8]:
```
!wget -O pose_landmarker.task -q https://storage.googleapis.com/
↪mediapipe-models/pose_landmarker/pose_landmarker_heavy/float16/1/
↪pose_landmarker_heavy.task
```

[9]:
```
!pip install torch-geometric
```

```
Collecting torch-geometric
  Downloading torch_geometric-2.6.1-py3-none-any.whl.metadata (63 kB)
                           63.1/63.1 kB
1.5 MB/s eta 0:00:00
Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-
packages (from torch-geometric) (3.11.10)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages
(from torch-geometric) (2024.10.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages
(from torch-geometric) (3.1.4)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages
(from torch-geometric) (1.26.4)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.10/dist-
packages (from torch-geometric) (5.9.5)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.10/dist-
packages (from torch-geometric) (3.2.0)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-
packages (from torch-geometric) (2.32.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages
(from torch-geometric) (4.67.1)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in
/usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (2.4.4)
Requirement already satisfied: aiosignal>=1.1.2 in
/usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (1.3.2)
Requirement already satisfied: async-timeout<6.0,>=4.0 in
/usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (4.0.3)
```

```
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-
packages (from aiohttp->torch-geometric) (24.3.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (0.2.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (1.18.3)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.10/dist-packages (from jinja2->torch-geometric) (3.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.10/dist-packages (from requests->torch-geometric) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-
packages (from requests->torch-geometric) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.10/dist-packages (from requests->torch-geometric) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.10/dist-packages (from requests->torch-geometric)
(2024.12.14)
Requirement already satisfied: typing-extensions>=4.1.0 in
/usr/local/lib/python3.10/dist-packages (from
multidict<7.0,>=4.5->aiohttp->torch-geometric) (4.12.2)
Downloading torch_geometric-2.6.1-py3-none-any.whl (1.1 MB)
                         1.1/1.1 MB
12.2 MB/s eta 0:00:00
Installing collected packages: torch-geometric
Successfully installed torch-geometric-2.6.1
```

[10]:
```python
import pandas as pd
import mediapipe as mp
from mediapipe.tasks import python
from mediapipe.tasks.python import vision
import cv2
import os
from google.colab.patches import cv2_imshow
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch_geometric.nn import GCNConv
from torch_geometric.data import Data
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
import pandas as pd
```

```
import torch
import os
import random
```

[11]:
```
import cv2
import os
import shutil
def read_frames_from_video():

    ranges = [420, 815, 1224, 1591, 1941, 2381, 2800, 3460, 3835, 4343, 4850,␣
 ↪5350, 5700, 6100, 6650, 6960, 7424, 7760, 8200]

    # List of ranges of frames to skip (e.g., skip frames from 50 to 100, and␣
 ↪200 to 250)
    skip_ranges = [(790, 815), (1211, 1233), (1920, 1990), (2348, 2428),␣
 ↪(2750,3000), (3370, 3490), (3805,3880), (4270, 4500), (4760, 4970), (5283,␣
 ↪5380), (5670, 5700), (5980, 6110), (6440, 6700), (6940, 7030), (7340, 7460),␣
 ↪(7730, 7785), (8120, 8250)]

    # Create the 'images' folder if it doesn't exist
    output_folder = "images"

    # Clear the 'images' folder if it exists, or create it
    if os.path.exists(output_folder):
        shutil.rmtree(output_folder)  # Remove the folder and its contents
    os.makedirs(output_folder, exist_ok=True)  # Create an empty folder

    vidcap = cv2.VideoCapture('d26arrd1_anon.mp4')
    success, image = vidcap.read()
    count = 0
    prefix_index = 0


    def is_skipped(frame_idx, skip_ranges):
        """Check if the frame index is within any of the skip ranges."""
        for start, end in skip_ranges:
            if start <= frame_idx <= end:
                return True
        return False

    while success:
        count += 1
        # Skip frames that fall within the skip_ranges
        if is_skipped(count, skip_ranges):
            success, image = vidcap.read()
            continue
```

```
        # Determine the current prefix based on the frame count
        while prefix_index < len(ranges) and count >= ranges[prefix_index]:
            prefix_index += 1

        # Create the filename with the appropriate prefix
        prefix = "{:02d}_".format(prefix_index)  # Start from 00
        filename = os.path.join(output_folder, "{}{:04d}.jpg".format(prefix,␣
↪count))

        # Save the frame
        cv2.imwrite(filename, image)
        success, image = vidcap.read()
```

[12]:
```
read_frames_from_video()
```

[13]:
```
print(os.listdir("/content/images")[-20:-1])
```

```
['08_3622.jpg', '15_6903.jpg', '19_8520.jpg', '11_5101.jpg', '05_2252.jpg',
'15_6808.jpg', '08_3752.jpg', '13_5861.jpg', '01_0544.jpg', '17_7726.jpg',
'12_5483.jpg', '18_7992.jpg', '18_7936.jpg', '02_0859.jpg', '19_8321.jpg',
'13_5798.jpg', '10_4652.jpg', '01_0451.jpg', '17_7494.jpg']
```

[14]:
```
from mediapipe import solutions
from mediapipe.framework.formats import landmark_pb2
import numpy as np


def draw_landmarks_on_image(rgb_image, detection_result):
  pose_landmarks_list = detection_result.pose_landmarks
  annotated_image = np.copy(rgb_image)

  # Loop through the detected poses to visualize.
  for idx in range(len(pose_landmarks_list)):
    pose_landmarks = pose_landmarks_list[idx]

    # Draw the pose landmarks.
    pose_landmarks_proto = landmark_pb2.NormalizedLandmarkList()
    pose_landmarks_proto.landmark.extend([
      landmark_pb2.NormalizedLandmark(x=landmark.x, y=landmark.y, z=landmark.z)␣
↪for landmark in pose_landmarks
    ])
    solutions.drawing_utils.draw_landmarks(
      annotated_image,
      pose_landmarks_proto,
      solutions.pose.POSE_CONNECTIONS,
      solutions.drawing_styles.get_default_pose_landmarks_style())
  return annotated_image
```

[16]:
```
base_options = python.BaseOptions(model_asset_path='pose_landmarker.task')
options = vision.PoseLandmarkerOptions(
    base_options=base_options,
    output_segmentation_masks=True)
detector = vision.PoseLandmarker.create_from_options(options)
df_array = []
image_directory = "/content/images"
files = os.listdir(image_directory)
for i, file_name in enumerate(files):
    if file_name.endswith(('.jpg', '.png', '.jpeg')):
        image_path = os.path.join(image_directory, file_name)
        image = mp.Image.create_from_file(image_path)

        detection_result = detector.detect(image)

        indices_to_keep = [24, 23, 12, 11, 13, 15, 14, 16]
        if i == 0:
          annotated_image = draw_landmarks_on_image(image.numpy_view(),␣
 ↪detection_result)
          annotated_image = cv2.cvtColor(annotated_image, cv2.COLOR_RGB2BGR)

          # Display the annotated image (or save it if needed).
          cv2_imshow(annotated_image)
          cv2.waitKey(0)
          cv2.destroyAllWindows()
        # Filter landmarks
        # print(detection_result.pose_landmarks[0])
        if len(detection_result.pose_landmarks) == 0: continue
        if len(detection_result.pose_landmarks[0]) != 33: continue
        detection_result.pose_landmarks[0] = [detection_result.
 ↪pose_landmarks[0][i] for i in indices_to_keep]

        # print(detection_result.pose_landmarks[0])

        # STEP 5: Process the detection result. In this case, visualize it.

        # Display the annotated image (or save it if needed).

        data = {
            # "Label": int(file_name[:2]),
            "Index": indices_to_keep,
            "X": [landmark.x for landmark in detection_result.
 ↪pose_landmarks[0]],
            "Y": [landmark.y for landmark in detection_result.pose_landmarks[0]]
        }
        df_array.append((pd.DataFrame(data), int(file_name[:2])))
```

[ ]:
```python
def cache_each_dataframe_to_csv(df_array, output_dir):
    """
    Saves each DataFrame in the df_array to a separate CSV file.

    :param df_array: List of tuples, where each tuple contains a DataFrame and an integer label.
    :param output_dir: Directory where the CSV files will be saved.
    """
    # Ensure the output directory exists
    os.makedirs(output_dir, exist_ok=True)

    for i, (df, label) in enumerate(df_array):
        # Create a unique filename for each DataFrame
        file_name = f"data_{label}_{i}.csv"
        file_path = os.path.join(output_dir, file_name)

        # Save the DataFrame to the CSV file
        df.to_csv(file_path, index=False)

        print(f"Saved DataFrame {i} with label {label} to {file_path}.")
cache_each_dataframe_to_csv(df_array, "/content/coordinates")
```

[18]:
```python
import zipfile
def zip_folder(folder_path, zip_file_path):
    """
    Compresses a folder into a ZIP file.

    :param folder_path: Path to the folder to be compressed.
    :param zip_file_path: Path to the output ZIP file.
    """
    # Ensure the folder exists
    if not os.path.exists(folder_path):
        print(f"Folder '{folder_path}' does not exist.")
        return

    # Create a ZIP file
    with zipfile.ZipFile(zip_file_path, 'w', zipfile.ZIP_DEFLATED) as zipf:
        # Walk through all files and subfolders in the folder
        for root, dirs, files in os.walk(folder_path):
            for file in files:
                # Full path to the file
                file_path = os.path.join(root, file)
                # Relative path for the ZIP archive
                arcname = os.path.relpath(file_path, folder_path)
                # Add file to the ZIP archive
                zipf.write(file_path, arcname)
```

```
    print(f"Folder '{folder_path}' successfully compressed into␣
 ↪'{zip_file_path}'.")
zip_folder("/content/coordinates", "/content/coordinates.zip")
```

```
Folder '/content/coordinates' successfully compressed into
'/content/coordinates.zip'.
```

[19]:
```
# Step 1: Generate Toy Data for Human Skeleton Motion
num_frames = 100  # Number of frames in the sequence
num_joints = 8  # Number of joints in the skeleton
num_features = 2  # (x, y) positions for each joint
num_classes = 20  # 3 classes for human actions (e.g., walk, run, jump)
# edge_index = torch.tensor([[16, 14, 12, 12, 24, 23, 11, 13],
#                            [14, 12, 11, 24, 23, 11, 13, 15]], dtype=torch.
 ↪long)
edge_index = torch.tensor([[7, 6, 5, 5, 1, 0, 2, 3],
                           [6, 5, 2, 1, 0, 2, 3, 4]], dtype=torch.long)
# edge_index = torch.tensor([[0, 1, 2, 3, 4, 5, 6, 7],
#                            [1, 2, 3, 4, 5, 6, 7, 0]], dtype=torch.long)
```

[20]:
```
def convert_to_tensors(data_frames):
    data_objects = []  # Initialize an empty list to store Data objects
    for df, label in data_frames:  # Unpack the DataFrame and label from each␣
 ↪element
        x_data = []

        for i in range(len(df["X"])):
            x_data.append((df["X"][i], df["Y"][i]))

        x_data = torch.tensor(x_data, dtype=torch.float32)
        y_data = torch.tensor([label], dtype=torch.long)

        # Create the PyTorch Geometric Data object
        data_object = Data(x=x_data, edge_index=edge_index, y=y_data)
        data_objects.append(data_object)
    return data_objects
```

[ ]:
```
# train_data_frames = convert_to_tensors(df_array)
```

[21]:
```
class STGCN(nn.Module):
    def __init__(self, in_channels, out_channels, num_joints, num_frames):
        super(STGCN, self).__init__()
        self.conv1 = GCNConv(in_channels, 64)  # First graph convolution layer
        self.conv2 = GCNConv(64, 128)  # Second graph convolution layer
        self.temporal_conv = nn.Conv2d(128, 128, kernel_size=(3, 1),␣
 ↪padding=(1, 0))  # Temporal convolution
```

```
        self.fc = nn.Linear(128 * num_frames * num_joints, out_channels)  # Fully connected layer for classification

    def forward(self, data):
        x, edge_index = data.x, data.edge_index
        # Graph convolution layers
        x = F.relu(self.conv1(x, edge_index))
        x = F.relu(self.conv2(x, edge_index))

        # Reshape x to include the time dimension (num_frames)
        # We will repeat the graph features across time frames
        x = x.unsqueeze(0)  # Add batch dimension: shape becomes [1, num_joints, 128]
        x = x.permute(0, 2, 1)  # Change shape to [1, 128, num_joints] (channels, joints)
        x = x.repeat(1, 1, num_frames)  # Repeat across time frames: [1, 128, num_joints, num_frames]

        # Apply the temporal convolution (across time)
        x = x.unsqueeze(3)  # Add time_steps dimension: [1, 128, num_joints, num_frames, 1]
        x = F.relu(self.temporal_conv(x))

        # Flatten the output for the fully connected layer
        x = x.view(x.size(0), -1)  # Flatten all but the batch dimension

        # Output classification
        x = self.fc(x)
        return x
```

[22]:
```
# Step 4: Training the ST-GCN

# Initialize the model, optimizer, and loss function
model = STGCN(in_channels=num_features, out_channels=num_classes, num_joints=num_joints, num_frames=num_frames)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()
```

[23]:
```
# Function to train the model with a sequence of frames
def train(model, data_frames, optimizer, criterion, epochs=100):
    model.train()
    for epoch in range(epochs):
        optimizer.zero_grad()
        total_loss = 0

        # Loop through the sequence of frames
```

```
        for frame in data_frames:
            # Forward pass for each frame in the sequence
            out = model(frame)

            # Compute loss for the current frame
            loss = criterion(out, frame.y)
            total_loss += loss

        # Backpropagation on the aggregated loss
        total_loss.backward()
        optimizer.step()

        if epoch % 20 == 0:
          print(f'Epoch {epoch+1}, Total Loss: {total_loss.item()}')



# Step 5: Evaluation

# def test(model, data_frame):
#     model.eval()
#     with torch.no_grad():
#         out = model(data_frame)  # Run the forward pass
#         pred = out.argmax(dim=1)  # Get the predicted class
#         correct = (pred == data_frame.y).sum()  # Compare with true label
#         accuracy = correct / len(data_frame.y)
#         return accuracy.item()
def test(model, test_data_frames):
    model.eval()
    total_correct = 0
    total_samples = 0

    with torch.no_grad():
        for data_frame in test_data_frames:
            # Run the forward pass for each data frame
            out = model(data_frame)
            pred = out.argmax(dim=1)  # Get the predicted class
            correct = (pred == data_frame.y).sum()  # Compare with true label
            total_correct += correct.item()
            total_samples += len(data_frame.y)

    accuracy = total_correct / total_samples
    return accuracy
```

[24]:
```
# Ustawienie proporcji podziału
test_ratio = 0.2
```

```
# Obliczenie liczby elementów w zbiorze testowym
total_data = len(df_array)
test_size = int(total_data * test_ratio)

# Podział danych
test_data_objects = df_array[:test_size]  # Pierwsze 20% danych jako zbiór␣
 ↪testowy
train_data_objects = df_array[test_size:]  # Pozostałe 80% danych jako zbiór␣
 ↪treningowy

# Konwersja ramek danych na obiekty Data
train_data_frames = convert_to_tensors(train_data_objects)
test_data_frames = convert_to_tensors(test_data_objects)
```

[26]:
```
train(model, train_data_frames, optimizer, criterion, epochs=100)

Test the model on the first frame (toy data)
accuracy = test(model, test_data_frames)
print(f'Accuracy on test data: {accuracy * 100:.2f}%')
```

```
Epoch 1, Total Loss: 541.756000
Epoch 2, Total Loss: 492.997960
Epoch 3, Total Loss: 448.628144
Epoch 4, Total Loss: 408.251611
Epoch 5, Total Loss: 371.508966
Epoch 6, Total Loss: 338.073159
Epoch 7, Total Loss: 307.646575
Epoch 8, Total Loss: 279.958383
Epoch 9, Total Loss: 254.762128
Epoch 10, Total Loss: 231.833537
Epoch 11, Total Loss: 210.968518
Epoch 12, Total Loss: 191.981352
Epoch 13, Total Loss: 174.703030
Epoch 14, Total Loss: 158.979757
Epoch 15, Total Loss: 144.671579
Epoch 16, Total Loss: 131.651137
Epoch 17, Total Loss: 119.802535
Epoch 18, Total Loss: 109.020307
Epoch 19, Total Loss: 99.208479
Epoch 20, Total Loss: 90.279716
Epoch 21, Total Loss: 82.154542
Epoch 22, Total Loss: 74.760633
Epoch 23, Total Loss: 68.032176
Epoch 24, Total Loss: 61.909280
Epoch 25, Total Loss: 56.337445
Epoch 26, Total Loss: 51.267075
Epoch 27, Total Loss: 46.653038
```

```
Epoch 28, Total Loss: 42.454265
Epoch 29, Total Loss: 38.633381
Epoch 30, Total Loss: 35.156377
Accuracy on test data: 77.41%
```